Day 167 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 24, Day 7 reference

Watch: IML - 04 Shapley - 03 SHAP (SHapley Additive exPlanation) Values (Statistical Learning and Data Science) | "Why Should I Trust You?" Explaining the Predictions of Any Classifier (KDD2016 video)

`day167_report_card.py`, segment by segment, in the order `main()` uses things. Today's new parts are fast_auc, coalition_values, shap_exact, shap_kernel, shap_linear and lime; everything else comes unchanged from the week's six exercise files (Days 161–166), plus boosting (Day 109) and ROC AUC (Day 117). There is no exercise file today. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: f = a model's log-odds as a function of a matrix · x = the row explained · bg = background rows · colfeat = the feature of each matrix column · v = coalition values · ϕ, phi = Shapley values · S = the dictionary every section shares

Segment 0 — Setup

Imports and constants

CODE

```
with contextlib.redirect_stdout(io.StringIO()):
    import day109_exercise as d109
    import day117_exercise as d117
    import day161_exercise as d161
    import day162_exercise as d162
    import day163_exercise as d163
    import day164_exercise as d164
    import day165_exercise as d165
    import day166_exercise as d166
FEAT = d161.NUM + d161.CAT
NF = len(FEAT)
THRESHOLD = 0.5
B = 1000
SIMS = 200
BG = 200
KERNEL_M = 500
LIME_N = 1000
N_EXPLAIN = 200
```

- `from math import factorial` — the one function needed from math: the Shapley weights.
- `FEAT = d161.NUM + d161.CAT` — the 12 feature names, in the tree matrix's column order.
- `BG = 200` — background rows; the first draft's 50 was too few (section 7 of the run).

Segment 1 — The parts

fast_auc: tied ranks with one sort

MATH

$AUC = (\text{sum of the positives' ranks} - n_1(n_1 - 1)/2) / (n_1\, n_0)$, ranks from 0

CODE

```
def fast_auc(y, p):
    u, inv, cnt = np.unique(p, return_inverse=True, return_counts=True)
    r = (np.cumsum(cnt) - cnt + (cnt - 1) / 2)[inv]
    n1 = y.sum(); n0 = len(y) - n1
    return float((r[y == 1].sum() - n1 * (n1 - 1) / 2) / (n1 * n0))
```

- `np.unique(p, return_inverse=True, return_counts=True)` — the distinct scores (sorted), each row's position among them, and how many rows share each.
- `np.cumsum(cnt) - cnt` — where each distinct score's block of ranks starts (from 0).
- `+ (cnt - 1) / 2` — the middle of the block: tied rows share the average rank, as in Day 117.
- `[inv]` — spreads each block's rank back to its rows.

ALL and coalition_values: every coalition, and its value

MATH

$v(S) = \text{mean over background rows } b \text{ of } f(x_S, b_{\text{not } S})$

CODE

```
ALL = ((np.arange(2 ** NF)[:, None] >> np.arange(NF)[None, :]) & 1).astype(bool)
def coalition_values(f, x, bg, masks, colfeat):
    out = np.empty(len(masks))
    for i in range(0, len(masks), 256):
        keep = masks[i:i + 256][:, colfeat]
        H = np.where(keep[:, None, :], x[None, None, :], bg[None, :, :])
        out[i:i + 256] = f(H.reshape(-1, H.shape[-1])).reshape(len(keep), len(bg)).mean(1)
    return out
```

- `np.arange(2 ** NF)[:, None] >> np.arange(NF)[None, :] & 1` — the bits of 0 to 4095: row k is coalition k, column j says whether feature j is in it.
- `keep = masks[i:i + 256][:, colfeat]` — indexing columns by colfeat copies each feature's flag to all of its matrix columns (one-hot features have several).
- `np.where(keep[:, None, :], x[None, None, :], bg[None, :, :])` — broadcasting to coalitions × background rows × columns: x where kept, the background row elsewhere.
- `H.reshape(-1, H.shape[-1])` — flattened to one big matrix, so the model is called once per batch.
- `.reshape(len(keep), len(bg)).mean(1)` — back to coalitions × background rows, then averaged over the background.

shap_exact: the definition, vectorised

MATH

$$\phi_j = \Sigma_{S \subseteq N \setminus \{j\}} |S|!\ (n - |S| - 1)!\ /\ n!\ [v(S \cup \{j\}) - v(S)]$$

CODE

```
def shap_exact(f, x, bg, colfeat):
    v = coalition_values(f, x, bg, ALL, colfeat)
    code = ALL @ (1 << np.arange(NF)); size = ALL.sum(1)
    wt = np.array([factorial(s) * factorial(NF - s - 1) / factorial(NF) for s in range(NF)])
    phi = np.zeros(NF)
    for j in range(NF):
        S = code[~ALL[:, j]]
        phi[j] = np.sum(wt[size[S]] * (v[S + (1 << j)] - v[S]))
    return phi, v[0], v[-1]
```

- `code = ALL @ (1 << np.arange(NF))` — each coalition's number: its flags times 1, 2, 4, ...; ALL[k] has code k.
- `factorial(s) * factorial(NF - s - 1) / factorial(NF)` — the Shapley weight for a coalition of size s that does not contain the feature.
- `S = code[~ALL[:, j]]` — the numbers of all coalitions without feature j.
- `v[S + (1 << j)] - v[S]` — adding 2^j to a coalition's number adds feature j: its marginal contribution, for all 2048 at once.
- `v[0], v[-1]` — the empty coalition (the background mean) and the full one (the prediction).

shap_kernel: sampled coalitions and a constrained regression

MATH

$$\text{minimise } \Sigma_z\ (v(z) - v_0 - \phi \cdot z)^2 \text{ subject to } \Sigma_j\ \phi_j = f(x) - v_0$$

CODE

```
def shap_kernel(f, x, bg, colfeat, M=KERNEL_M, seed=0):
    rng = np.random.RandomState(seed)
    sizes = np.arange(1, NF); pk = (NF - 1) / (sizes * (NF - sizes)); pk = pk / pk.sum()
    Z = np.zeros((2 * M, NF), bool)
    for i in range(M):
        Z[2 * i, rng.choice(NF, rng.choice(sizes, p=pk), replace=False)] = True
        Z[2 * i + 1] = ~Z[2 * i]
    v = coalition_values(f, x, bg, np.vstack([np.zeros((1, NF), bool), np.ones((1, NF), bool),
        Z]), colfeat)
    v0, D, vz = v[0], v[1] - v[0], v[2:]
    A = Z[:, :-1].astype(float) - Z[:, -1:].astype(float)
    phi = np.linalg.lstsq(A, vz - v0 - Z[:, -1] * D, rcond=None)[0]
    return np.r_[phi, D - phi.sum()], v0, v[1]
```

- `pk = (NF - 1) / (sizes * (NF - sizes))` — the Shapley kernel's total weight for each coalition size; sizes are drawn in proportion to it.
- `Z[2 * i + 1] = ~Z[2 * i]` — every sampled coalition is paired with its complement.
- `np.vstack([np.zeros(...), np.ones(...), Z])` — the empty and full coalitions first, so one call gives v0 and f(x) too.
- `Z[:, :-1] - Z[:, -1:]` — substituting $\phi_{last} = D$ – the others' sum, so the values add up exactly.
- `np.linalg.lstsq(A, t, rcond=None)[0]` — ordinary least squares; the kernel weights are already in how often each size was drawn.

shap_linear and lime

MATH

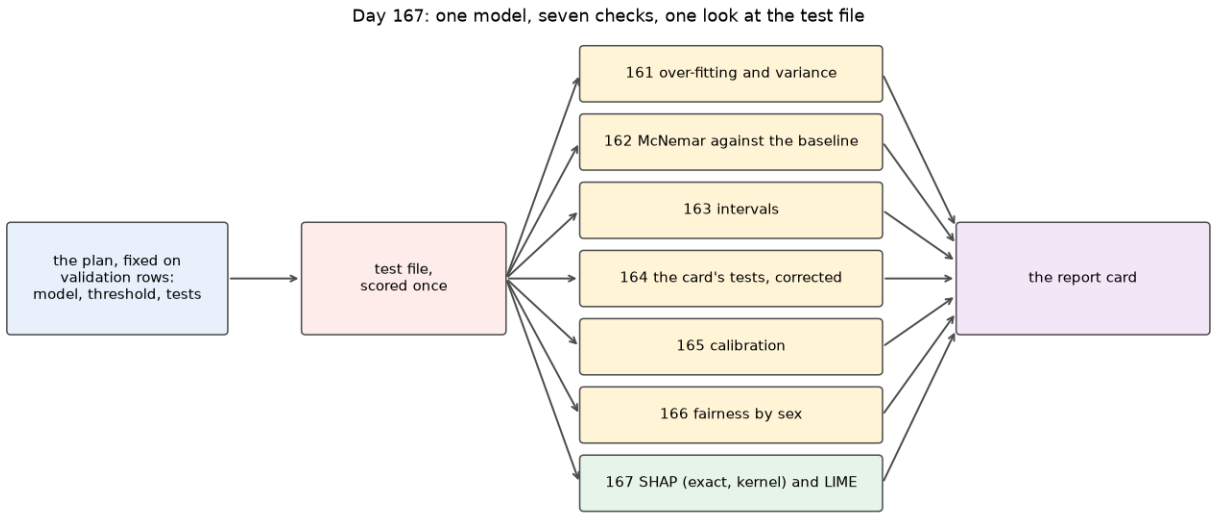
$$\beta = (A^T W A + R)^{-1} A^T W f, \quad W = diag(\pi)$$

CODE

```
def shap_linear(w, x, bg, colfeat):
    return np.bincount(colfeat, w[1:] * (x - bg.mean(0)), NF)
def lime(f, x, bg, colfeat, N=LIME_N, width=None, seed=0):
    rng = np.random.RandomState(seed); width = width or 0.75 * np.sqrt(NF)
    Z = rng.rand(N, NF) < 0.5; Z[0] = True
    H = np.where(Z[:, colfeat], x[None, :], bg[rng.randint(0, len(bg), N)])
    wt = np.exp(-(~Z).sum(1) / width ** 2)
    A = np.c_[np.ones(N), Z.astype(float)]
    R = 1e-3 * np.eye(NF + 1); R[0, 0] = 0.0
    return np.linalg.solve(A.T @ (A * wt[:, None]) + R, A.T @ (wt * f(H)))[1:]
```

- `np.bincount(colfeat, w[1:] * (x - bg.mean(0)), NF)` — with weights: each column's contribution summed into its feature's slot.
- `width = width or 0.75 * np.sqrt(NF)` — or returns the first truthy value: the default width when none is given.
- `Z[0] = True` — row 0 keeps every feature: the row itself.
- `bg[rng.randint(0, len(bg), N)]` — one random background row per sample.
- `np.exp(-(~Z).sum(1) / width ** 2)` — the number of changed features is the squared distance; closer samples weigh more.
- `A.T @ (A * wt[:, None]) + R` — the weighted normal equations, with a tiny ridge R; [1:] drops the intercept.

SYSTEM DIAGRAM



The plan, one look at the test file, and the seven checks.

Segment 2 — The experiments

train_models: models as a dictionary of functions

CODE

```
M = {"logistic": (lambda X: d161.predict_logistic(w, X), "o"), "boosting": (lambda X:
    d109.sigmoid(d109.predict_boost(m, X)), "t")}
return dict(tr=tr, te=te, fi=fi, va=va, ref=ref, y=y, Xt=Xt, Xo=Xo, w=w, m=m, M=M)
```

- `lambda X: d161.predict_logistic(w, X)` — a function that remembers w: later code calls every model the same way.
- `(..., "o")` — each model is paired with the matrix it needs: one-hot or tree.
- `dict(tr=tr, te=te, ...)` — keyword arguments build the dictionary S that every section reads and adds to.

demo_tests: four rates by sex as two-proportion tests

CODE

```
for mt, base in (("sel", np.ones(len(yt), bool)), ("tpr", yt == 1), ("fpr", yt == 0), ("ppv", d)):
    hit = yt == 1 if mt == "ppv" else d
    _, _, pg, _, _ = d162.two_prop_test(int(np.sum(hit & base & mn)), int(np.sum(base & mn)),
        int(np.sum(hit & base & w)), int(np.sum(base & w)))
    names.append(f"{mt} gap by sex"); p.append(pg)
```

- `(("sel", np.ones(len(yt), bool)), ("tpr", yt == 1), ...)` — each rate is a share of some base set of rows: everyone, the positives, the negatives, the selected.
- `hit = yt == 1 if mt == "ppv" else d` — what counts as a hit: a true positive for precision, otherwise being selected.
- `hit & base & mn` — three boolean masks combined: hits within the base set among men.

demo_shap: a helper lambda and the quick first-draft check

CODE

```
pick = lambda seed, k: S["fi"][np.random.RandomState(seed).choice(len(S["fi"]), k, replace=False)]
bgi = pick(5, BG); bgT, bgO = S["Xt"][bgi], S["Xo"][bgi]
rows = np.random.RandomState(6).choice(len(S["te"]["y"]), N_EXPLAIN, replace=False)
fl = lambda X: S["w"][0] + X @ S["w"][1:]
fb = lambda X: d109.predict_boost(S["m"], X)
S.update(OH=OH, bgT=bgT, bgO=bgO, rows=rows, fl=fl, fb=fb)
cg = S["tr"]["capital-gain"][bi].astype(float)
```

- `pick = lambda seed, k: ...` — a one-line function: k distinct fitting rows from a seeded generator.
- `S.update(OH=OH, bgT=bgT, ...)` — adds several keys to the shared dictionary at once.
- `.astype(float)` — Adult's columns are text; capital gains are converted to count and compare them.

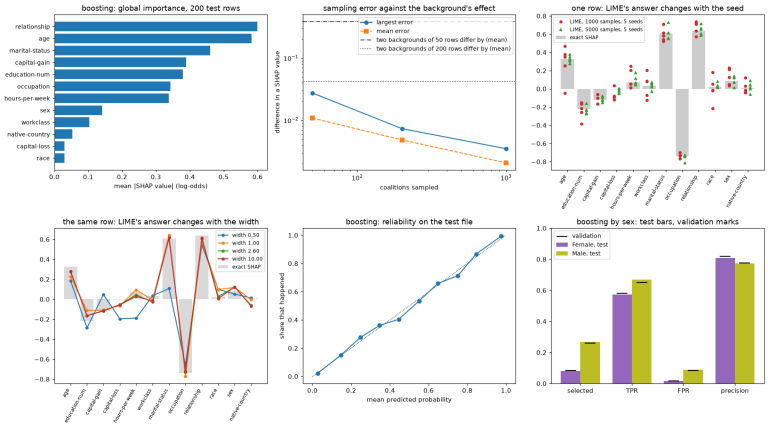
demo_lime: comparing top-three sets

CODE

```
ov.append(len(set(np.argsort(-np.abs(L))[:3]) & set(np.argsort(-np.abs(P))[:3])))
cr.append(np.corrcoef(L, P)[0, 1])
```

- `np.argsort(-np.abs(L))[:3]` — positions of the three largest absolute values.
- `set(...) & set(...)` — set intersection: the features both methods put in their top three.
- `np.corrcoef(L, P)[0, 1]` — the off-diagonal entry of the 2 × 2 correlation matrix.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.unique(..., return_counts=True)` — distinct values and how often each occurs.
- `a >> b & 1, 1 << j` — bit operations: bit b of a; the number 2^j.
- `np.linalg.lstsq(A, t, rcond=None)` — least-squares solution of A · x = t.
- `x or default` — the first truthy value.
- `dict.update(k=v, ...)` — add or replace several keys.
- `set(a) & set(b)` — items in both.

End of Day 167 syntax reference. For the experiments and what they showed, see the Day 167 complete guide.